UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 8 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (8).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Classification of parallel processors is constructed according to the relationship between the instruction stream and the data stream, as proposed in Flynn's taxonomy. A parallel processor is described by whether it issues one or many instruction streams and whether it operates on one or many data streams, giving four categories: SISD (single instruction, single data), SIMD (single instruction, multiple data), MISD (multiple instruction, single data) and MIMD (multiple instruction, multiple data). This construction classifies architectures by their parallelism at the instruction and data level.

(b) One example for each parallel processor architecture: SISD — a conventional single-core von Neumann processor (e.g., a classic uniprocessor PC); SIMD — a vector processor or GPU (e.g., Intel SSE/AVX vector unit, or a GPU executing the same instruction on many data elements); MISD — a systolic array or a fault-tolerant system with replicated processors operating on the same data stream (e.g., a pipeline of processors where each stage receives the same data in sequence); MIMD — a multicore processor or a cluster of workstations (e.g., Intel Core i7 multicore, or a distributed-memory multiprocessor).

Unofficial marking guide:

- *1 mark: State that classification is based on instruction stream and data stream (Flynn's taxonomy).*
- *1 mark: Name the four categories (SISD, SIMD, MISD, MIMD) correctly.*
- *1 mark: Give a correct example for SISD.*
- *1 mark: Give a correct example for SIMD.*
- *1 mark: Give a correct example for MIMD (MISD example may be accepted if correct; full 3 marks require three distinct correct examples).*

Model confidence: high.

Model note: Assumes the question refers to Flynn's taxonomy, which is the standard classification for parallel processors in computer organization. Examples are illustrative; any valid example for each category is acceptable.

QUESTION A2 (5 marks)

Topic: Computer generations and history

Source: CSC429 (5).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) AI is introduced in the fifth generation of computers (the present/future generation), which is associated with artificial intelligence, parallel processing and natural-language interfaces.

(b) The programming language used for this technology is Prolog (a logic programming language widely used for AI), with LISP also accepted as a classical AI language.

(c) The advantage of embedding AI in everyday life is that it automates routine and complex tasks, making systems smarter, faster and more convenient — for example smart homes adjust lighting/heating automatically, intelligent tutoring systems personalise learning, and robots perform dangerous or repetitive work — thereby saving time, improving efficiency and raising quality of life.

(d) One scenario where AI can harm people: an autonomous vehicle or robot whose AI misclassifies a pedestrian or obstacle (e.g. fails to recognise a person at night) could cause a fatal accident; similarly, a poorly trained AI in a medical diagnosis or smart-home device could give a wrong decision that injures users. This shows AI can harm people when its training data is biased/incomplete or when it acts without adequate human oversight.

Unofficial marking guide:

- *1 mark: (a) Fifth generation (AI generation) correctly named.*
- *1 mark: (b) Prolog (or LISP) named as the AI programming language.*
- *1 mark: (c) Any valid advantage of embedding AI in everyday life, e.g. automation of tasks, improved efficiency/convenience, personalisation, safety in dangerous work.*
- *1 mark: (d) A clear, plausible scenario in which AI harms people (e.g. autonomous vehicle accident, robot injury, faulty AI medical decision).*
- *1 mark: (d) Explanation of how/why the harm occurs (e.g. misclassification, biased/incomplete training data, lack of*

© Hak Cipta Universiti Teknologi MARA

human oversight).
Model confidence: medium.
Model note: OCR errors corrected: 'A1' read as 'AI'. The paper does not specify a single language; Prolog is the standard fifth-generation/AI answer, with LISP as an acceptable alternative. Part (d) is marked out of 2 as indicated.

QUESTION A3 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The technique is direct addressing (also called absolute addressing). The instruction's address field A contains the actual/effective address of the operand, so the operand is fetched directly from memory location A — there is no intermediate address calculation or pointer dereference.

(b) How it is used: The instruction is fetched and its opcode decoded; the CPU takes the address field A as the effective address (EA = A) and places it on the address bus. The memory is read at location A, and the operand retrieved is passed to the ALU/register for execution of the instruction. For example, with LOAD A, the contents of memory word A are loaded into the accumulator. The address A is fixed at assembly/compile time, so the operand's location must be known when the program is written.

(c) Advantage: No address calculation is required — the effective address is obtained immediately from the instruction, so operand fetch is fast and simple (only one memory access for the operand). Disadvantage: The address field must be large enough to hold a full memory address, making the instruction long; also the address is fixed, so the technique offers limited flexibility (no relocation/indirection, restricted addressable range).

Unofficial marking guide:

- *1 mark: Correctly names the technique as direct addressing (absolute addressing).*
- *1 mark: States that the address field A is the effective address of the operand (EA = A).*
- *1 mark: Explains the fetch/execute step — address A placed on address bus, memory read at A, operand used by the instruction (e.g. LOAD A).*
- *1 mark: Gives one valid advantage (e.g. no address calculation, fast/simple operand fetch, only one memory reference).*
- *1 mark: Gives one valid disadvantage (e.g. long instruction/limited address range, fixed address so no flexibility/relocation).*

Model confidence: high.
Model note: Figure 2 is inferred from the extracted text (Instruction: Opcode | Address A, then Memory: Operand), which matches direct addressing. If the figure actually shows an arrow from Address A to a memory word containing a further address, the answer would instead be indirect addressing; the text as given supports direct addressing.

QUESTION A4 (5 marks)

Topic: Data representation and number systems

Answer: 3.2510 = 0x40500000 (sign=0, exponent=10000000, fraction=10100000000000000000000)

- *1 mark for the sign bit.*
- *1 mark for the biased exponent.*
- *1 mark for the 23-bit fraction.*
- *1 mark for the correct 32-bit grouping.*
- *1 mark for the final hexadecimal value.*

QUESTION A5 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (11).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The non-maskable interrupt (NMI) is message (b): "A hardware failure has been found. Please restart your computer now." Justification: an NMI is reserved for catastrophic events such as a hardware failure, power failure or memory error; it cannot be disabled or ignored by the processor (it is not maskable), so the system must respond immediately, typically by halting or restarting. Message (a), the business-class upgrade offer, is an ordinary, deferrable event generated by the web/booking application; it can be ignored or postponed by the user and is therefore a maskable interrupt.

(b) A maskable interrupt can be temporarily disabled (masked) by the processor, for example by clearing

© Hak Cipta Universiti Teknologi MARA

the interrupt-enable flag or by an interrupt mask register, so its servicing can be delayed until the CPU is ready; it is used for normal I/O and software events. A non-maskable interrupt cannot be disabled by the processor; it is always recognised and is used for urgent fault conditions such as hardware failure, so that critical events are never lost.

(c) The common way to handle multiple interrupts is by assigning priorities and using an interrupt controller with a priority scheme. When several interrupts arrive, the CPU compares their priorities: a higher-priority interrupt can preempt (interrupt) the servicing of a lower-priority one, while a lower-priority request waits until the current handler finishes. In the scenario, the hardware-failure NMI (b) has the highest priority and is serviced immediately, preempting the normal booking/upgrade interrupt (a); the upgrade message is held pending and handled after the critical condition is dealt with. This is typically implemented by the interrupt controller asserting the highest-priority request and the CPU saving the current context before branching to the appropriate interrupt service routine.

Unofficial marking guide:

- *1 mark: identifying message (b) as the non-maskable interrupt*
- *1 mark: stating that an NMI cannot be masked/disabled by the processor*
- *1 mark: linking message (b) to a critical hardware-failure condition requiring immediate action*
- *1 mark: explaining that message (a) is a normal/deferrable maskable interrupt*
- *1 mark: defining maskable interrupt as one that can be disabled/delayed*
- *1 mark: defining non-maskable interrupt as one that cannot be disabled and is for urgent events*
- *1 mark: stating that multiple interrupts are handled by priority assignment*
- *1 mark: explaining that a higher-priority interrupt can preempt a lower-priority one*
- *1 mark: applying priority to the scenario (hardware-failure NMI serviced first, upgrade interrupt deferred)*
- *1 mark: mentioning the interrupt controller / saving context / ISR as the handling mechanism*

Model confidence: high.

Model note: Figure 1 is represented only by its extracted text; the labels (a) and (b) are assumed to correspond to the upgrade message and the hardware-failure message respectively, as shown in the question text. The answer is an unofficial model answer, not an official marking scheme.

QUESTION A6 (10 marks)

Topic: Data representation and number systems

a) Answer: left 63 -> 0 0111111; right -41 -> 1 0101001; 63 - (-41) = 104 -> sign-magnitude 0 1101000

- *1 mark for operand representations.*
- *1 mark for sign handling.*
- *1 mark for magnitude arithmetic.*
- *1 mark for the result sign bit.*
- *1 mark for the final byte.*

b) Answer: left 63 -> 1's 00111111, 2's 00111111; right -41 -> 1's 11010110, 2's 11010111; 63 - (-41) = 104 -> 1's 01101000, 2's 01101000

- *1 mark for the positive operand in binary.*
- *1 mark for the 1's complement of the negative operand.*
- *1 mark for adding 1 to form the 2's complement.*
- *1 mark for the correct 8-bit result.*
- *1 mark for converting the result back to decimal.*

QUESTION A7 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (6).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Computer architecture refers to those attributes of a system that are visible to a programmer — the conceptual design and functional behaviour of the machine. It includes the instruction set, the number and types of data representations (e.g. word length), addressing modes, registers visible to the programmer, and I/O mechanisms. Architecture answers the question 'what does the machine do?' and is largely independent of the underlying hardware implementation. Computer organization, by contrast, refers to the operational units and their interconnections that realize the architectural specification — the physical/logical implementation. It includes the control signals, the memory technology used, the bus structure, the clock rate, the microarchitecture of the datapath, and the interfacing of peripherals. Organization answers 'how is it implemented?' and can differ between machines that share the same

© Hak Cipta Universiti Teknologi MARA

architecture (e.g. two processors with the same instruction set but different cache sizes, pipeline depths or clock speeds). In short, architecture is the programmer-visible functional view, while organization is the implementation view; many different organizations can realize one architecture.

(b) Three CPU components and their functions:
1. Control Unit (CU): fetches and decodes instructions and generates the timing and control signals that direct all other units (ALU, registers, memory, I/O) to carry out each instruction. It sequences the fetch–decode–execute cycle and coordinates data movement.
2. Arithmetic Logic Unit (ALU): performs the arithmetic operations (add, subtract, multiply, etc.) and logical operations (AND, OR, NOT, XOR, shifts, comparisons) on data held in registers; it also sets condition/status flags (zero, carry, sign, overflow) used for branching.
3. Registers (e.g. Program Counter, Instruction Register, Accumulator/general-purpose registers): small, very fast storage inside the CPU. The Program Counter holds the address of the next instruction, the Instruction Register holds the instruction currently being executed, and general-purpose/accumulator registers hold operands and intermediate results. They provide the fastest storage for the working data of the CPU.
(Other acceptable components: cache, bus interface, clock, MAR/MBR.)

Unofficial marking guide:

• 1 mark: correct definition of architecture as programmer-visible attributes (instruction set, data types, addressing modes, registers)
• 1 mark: correct definition of organization as implementation/operational units and their interconnection (control signals, memory technology, bus, clock)
• 1 mark: clear contrast — architecture = 'what it does', organization = 'how it is implemented'
• 1 mark: valid example/illustration (e.g. same architecture realized by different organizations)
• 1 mark: first CPU component correctly named
• 1 mark: correct function of first component
• 1 mark: second CPU component correctly named
• 1 mark: correct function of second component
• 1 mark: third CPU component correctly named
• 1 mark: correct function of third component
Model confidence: high.
Model note: No figure or table required; question is fully answerable from text. Marks split as given: (a) 4 marks, (b) 6 marks (2 marks per component).

QUESTION A8 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (12).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two functions of an I/O module (4 marks):

Function 1 — Processor/device communication and control (interface/translation): The I/O module acts as the intermediary between the CPU and external devices. It receives control commands from the CPU (e.g. read, write, status request) and translates them into signals the device understands, and conversely converts device signals/data into a form the CPU can use. It also reports device status (ready, busy, error) back to the processor, so the CPU does not need to know device-specific details.

Function 2 — Data buffering and data transfer (synchronization): The I/O module temporarily holds data in buffers/registers to compensate for the speed mismatch between the fast CPU/system bus and the much slower peripheral. It handles the actual transfer of data between the device and the CPU or memory (e.g. via DMA), and performs error detection (e.g. parity) on the transferred data. This frees the CPU from managing timing differences and lets it continue other work while the module completes the transfer.

(b) Two categories of external I/O devices with examples (4 marks):

Category 1 — Human-readable (human–machine interface) devices: devices that communicate directly with a human user. Example: keyboard, mouse, monitor/display, printer.

Category 2 — Machine-readable devices: devices that communicate with other machines/equipment rather than a person. Example: disk drive, magnetic tape drive, sensor/actuator, network interface card.

(The third standard category, for completeness, is communication devices, e.g. modem/NIC.)

(c) I/O techniques (2 marks):

© Hak Cipta Universiti Teknologi MARA

Requires active CPU intervention: Programmed I/O (polling) — the CPU repeatedly checks the device status and moves data itself. (Interrupt-driven I/O also involves CPU servicing each transfer.)

Does not require active CPU intervention: Direct Memory Access (DMA) — the DMA controller transfers data directly between device and memory, interrupting the CPU only when the whole block is complete.

Unofficial marking guide:

• 1 mark: correct identification of a first function of an I/O module (e.g. processor/device communication and control)
• 1 mark: discussion/explanation of that first function
• 1 mark: correct identification of a second function (e.g. data buffering/synchronization or error detection)
• 1 mark: discussion/explanation of that second function
• 1 mark: correct naming of a first device category (e.g. human-readable)
• 1 mark: appropriate example for that category (e.g. keyboard/monitor)
• 1 mark: correct naming of a second device category (e.g. machine-readable) with an appropriate example (e.g. disk drive/sensor)
• 1 mark: correct naming of an I/O technique requiring active CPU intervention (e.g. programmed I/O/polling)
• 1 mark: correct naming of an I/O technique not requiring active CPU intervention (e.g. DMA)

Model confidence: high.
Model note: Question text has minor OCR/grammar issues ('each categories'); interpreted as 'each category'. Part (c) asks only to name techniques, so brief naming suffices; the parenthetical explanations are added for clarity. The third device category (communication devices) is mentioned only as supplementary information and is not required for marks.

QUESTION A9 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (16).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a)(i) Function — System interconnection: the system bus (front-side/memory-I/O bus) is the shared set of communication pathways that links the CPU, main memory and I/O controllers so that data, addresses and control signals can be exchanged between all major system components; it is the general-purpose backbone of the whole computer. CPU interconnection: the CPU-internal (or CPU-local) interconnection links the components inside the processor — the control unit, ALU, registers and cache — and carries the internal data/address/control traffic needed to fetch, decode and execute instructions. In short, system interconnection connects the processor to the rest of the machine, whereas CPU interconnection connects the functional units within the processor.

(a)(ii) Physical appearance — System interconnection: appears as a set of parallel conducting tracks (bus lines) on the motherboard, with expansion slots/connectors into which memory modules and I/O adapter cards plug; it is relatively long, exposed on the board and shared by many devices. CPU interconnection: appears as very short, dense on-chip wiring (metal interconnect layers) inside the CPU chip, or a compact local bus on the processor package; it is not exposed to the user and is much shorter and faster than the system bus.

(b) The three bus types are the data bus, the address bus and the control bus. Data bus: a bidirectional set of lines that carries the actual data (and instructions) between the CPU, memory and I/O; its width determines how many bits can be transferred at once and hence affects performance. Address bus: a unidirectional set of lines that carries the address of the memory location or I/O port being accessed, from the CPU outwards; its width determines the maximum addressable memory (e.g. n lines → 2^n locations). Control bus: a set of lines carrying timing and command signals (e.g. read/write, memory/IO select, interrupt request/acknowledge, clock, bus grant/request) that coordinate and synchronise the transfer of data and addresses between the components.

Unofficial marking guide:

• 1 mark: correct function of system interconnection (links CPU, memory and I/O; system-wide backbone)
• 1 mark: correct function of CPU interconnection (links internal CPU units such as ALU, registers, control unit, cache)
• 1 mark: correct physical appearance of system interconnection (motherboard tracks/expansion slots, longer, shared, exposed)
• 1 mark: correct physical appearance of CPU interconnection (short on-chip interconnect/local bus, compact, not user-visible)
• 1 mark: data bus — carries data/instructions between CPU, memory and I/O (bidirectional)
• 1 mark: data bus — width determines number of bits transferred at once / affects performance
• 1 mark: address bus — carries address of memory location or I/O port (unidirectional, CPU outwards)

© Hak Cipta Universiti Teknologi MARA

• 1 mark: address bus — width determines maximum addressable memory (2^n)
• 1 mark: control bus — carries command/timing signals (read/write, memory/IO select, interrupt, clock)
• 1 mark: control bus — synchronises/coordinates transfers between components
Model confidence: high.
Model note: Question text is clear; no OCR correction needed. The comparison assumes the standard CSC429 treatment of system bus vs CPU-internal interconnection; if the course uses 'CPU interconnection' to mean a dedicated processor-to-cache/processor-to-processor local bus, the same functional/physical distinction still applies.

QUESTION A10 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (17).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Three types of replacement algorithm [6 marks]

1. Random replacement: When a cache line must be evicted, a line is chosen at random (often via a simple pseudo-random counter). It requires almost no bookkeeping hardware, so it is cheap and fast, but it ignores locality and may evict a line that is about to be reused, giving poorer hit ratios than LRU-style schemes.

2. First-In-First-Out (FIFO): Lines are evicted in the order they were brought into the set — the oldest line is replaced first, maintained by a circular pointer/queue per set. It is simple to implement, but it does not track usage, so a frequently used old line can still be evicted (no account of temporal locality), and it can suffer Belady's anomaly in principle.

3. Least Recently Used (LRU): The line that has gone longest without being referenced is evicted, exploiting temporal locality. It gives good hit ratios but needs per-line usage information (e.g. counters or a reference/age bit stack), so it is costly in hardware and difficult to implement exactly for high associativity; approximations such as pseudo-LRU or the clock/second-chance algorithm are often used instead.

(Other acceptable answers: Least Frequently Used (LFU) — evict the line with fewest references; Optimal/OPT (Belady's) — evict the line used furthest in the future, used only as a theoretical benchmark since it needs future knowledge.)

(b) Two types of mapping requiring a replacement algorithm [2 marks]

Fully associative mapping and set-associative mapping. (Direct mapping needs no replacement algorithm because each block has exactly one possible cache line.)

(c) Reason the replacement algorithm must be implemented [2 marks]

In associative and set-associative caches a new block can be placed in any line of a set (or of the whole cache), so when that set is full a choice must be made about which existing line to evict. The replacement algorithm makes this choice; a good policy (e.g. LRU) evicts the least useful line and so raises the hit ratio and reduces average memory access time, while a poor policy causes useful blocks to be discarded and increases misses.

Unofficial marking guide:

• 1 mark: correct identification/description of Random replacement
• 1 mark: correct identification/description of FIFO replacement
• 1 mark: correct identification/description of LRU replacement
• 3 marks: elaboration of each of the three algorithms (1 mark each for explaining how the victim is chosen and its advantage/disadvantage)
• 1 mark: fully associative mapping listed
• 1 mark: set-associative mapping listed
• 1 mark: explains that in associative/set-associative mapping a block can go into more than one line, so a victim must be chosen when the set is full
• 1 mark: links a good replacement policy to improved hit ratio / reduced average memory access time (or the cost of a poor policy)
Model confidence: high.
Model note: Question text is clear; no OCR correction needed. Part (a) accepts any three standard algorithms (Random, FIFO, LRU, LFU, Optimal); the model answer uses the three most common. Mark allocation assumes 2 marks per algorithm in (a) (1 identification + 1 elaboration).

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (18).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption: Figure 2 is interpreted as y = A - ((2C * D) / (B + A^2)). The expression is evaluated as: temp1 = 2*C; temp2 = temp1*D; temp3 = A*A; temp4 = B + temp3; temp5 = temp2 / temp4; y = A - temp5. Registers A, B, C, D are fixed; x, y, z are temporary. Final result must be in y.

One-address program (accumulator-based):

```
LOAD A       ; ACC = A
MUL A        ; ACC = A^2
ADD B        ; ACC = A^2 + B
STOR z       ; z = A^2 + B
LOAD C       ; ACC = C
MUL D        ; ACC = C*D
ADD C        ; ACC = 2*C*D  (since C*D + C*D = 2*C*D)
DIV z        ; ACC = (2*C*D)/(A^2+B)
STOR x       ; x = (2*C*D)/(A^2+B)
LOAD A       ; ACC = A
SUB x        ; ACC = A - x
STOR y       ; y = final answer
```

Two-address program (operand1 = operand1 op operand2):

```
MOV x, C     ; x = C
ADD x, C     ; x = 2C
MUL x, D     ; x = 2C*D
MOV z, A     ; z = A
MUL z, A     ; z = A^2
ADD z, B     ; z = B + A^2
DIV x, z     ; x = (2C*D)/(B + A^2)
MOV y, A     ; y = A
SUB y, x     ; y = A - x
```

Three-address program (dest = src1 op src2):

```
MUL x, C, D   ; x = C*D
ADD x, x, x   ; x = 2*C*D
MUL z, A, A   ; z = A^2
ADD z, B, z   ; z = B + A^2
DIV x, x, z   ; x = (2*C*D)/(B + A^2)
SUB y, A, x   ; y = A - x
```

Unofficial marking guide:

- *1 mark: correct interpretation of Figure 2 as y = A - ((2C*D)/(B + A^2)) and correct order of operations.*
- *6 marks: one-address program – 1 mark for correct use of LOAD/STOR, 1 mark for computing A^2+B, 1 mark for computing 2C*D, 1 mark for division, 1 mark for final subtraction, 1 mark for storing final answer in y.*
- *6 marks: two-address program – 1 mark for MOV to initialise, 1 mark for computing 2C, 1 mark for 2C*D, 1 mark for A^2+B, 1 mark for division, 1 mark for final subtraction into y.*
- *6 marks: three-address program – 1 mark for MUL x,C,D, 1 mark for doubling to 2C*D, 1 mark for MUL z,A,A, 1 mark for ADD z,B,z, 1 mark for DIV x,x,z, 1 mark for SUB y,A,x.*
- *1 mark: correct assembly notation (mnemonics and operand order) consistent with Table 1 for all three programs.*

Model confidence: medium.

*Model note: Figure 2 is not fully legible from the extracted text; the expression is assumed to be y = A - (2C*D)/(B + A^2). If the figure shows a different grouping (e.g., denominator B + A^2 or (B+A)^2), the programs must be adjusted accordingly. Table 1 uses inconsistent punctuation (e.g., ADD (p.q.r)); operand order is taken as dest, src1, src2 for three-address and dest, src for two-address. One-address instructions are assumed to use an implicit accumulator.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA